

**20472: Software Engineering**

Spring 2026 • Instructor: Prof. Soomin Kim

**Manual Testing of grep 2.0**

Due: 2026-09-28 23:59

## 1 과제의 목적

이 과제를 통해 여러분들은 주어진 프로그램 `grep.c`를 효과적으로 테스트하는 입력을 직접 설계해야 합니다. 테스트의 품질은 "몇 개를 돌렸는가"가 아니라 "프로그램의 얼마나 넓은 부분을 실행시켜보았는가"로 이야기합니다. 이 과제에서는 30개 이하의 실행만으로 프로그램의 코드를 최대한 넓게 탐색하는 것이 목표입니다. 입력 개수가 제한되어 있으므로, 무작정 많이 만드는 전략은 통하지 않습니다. 프로그램이 어떤 입력에 대해 어떤 경로를 타는지 예측하고, 서로 다른 경로를 노리는 입력을 골라야 합니다.

## 2 대상 프로그램

대상 프로그램은 GNU grep 2.0 (1992년(!) 버전)을 하나의 C 소스코드 파일로 합친 `grep.c`입니다.

소스코드를 제공하기 때문에 소스코드를 직접 읽고 정적으로 분석해도 좋고, 읽지 않고 동적으로 실행시켜 가면서 블랙박스 접근해도 좋습니다.

### 2.1 입력 형식

프로그램의 입력 종류는 다음의 세 가지입니다.

- (a) 옵션: CLI 인자로 제공
- (b) 패턴: CLI positional argument, `-e` 패턴, 또는 `-f` 패턴\_파일 로 제공
- (c) 대상 텍스트: CLI 인자로 제공한 파일 경로, 또는 `stdin`으로 제공

따라서 프로그램의 실행은 다음과 같은 형태를 가집니다. 당연히 다양한 옵션을 더 추가하는 것도 가능합니다.

- `grep "패턴" 파일.txt`
  - 패턴을 CLI positional argument로 제공, 대상 텍스트를 파일로 제공
- `cat 파일.txt | grep "패턴"`
  - 패턴을 CLI positional argument로 제공, 대상 텍스트를 `stdin`으로 제공
- `grep -e "패턴1" -e "패턴2" 파일.txt`
  - 패턴을 `-e` 옵션으로 여러 개 제공, 대상 텍스트를 파일로 제공
- `grep -f 패턴_파일.txt 파일.txt`
  - 패턴을 `-f` 옵션으로 제공, 대상 텍스트를 파일로 제공

### 3 과제 내용물

이 문서와 함께 배포되는 파일들은 다음과 같은 구조를 갖습니다. Linux 환경에서 make 명령어를 통해 빌드할 수 있으며, Linux 환경이 없는 경우 WSL2, Docker 등의 환경을 통해 Linux를 설치한 뒤 빌드할 수 있습니다.

```

grep-starter/
├── hw1.pdf           ← 이 문서
├── Makefile
├── src/
│   ├── grep.c       ← 대상 프로그램
│   └── *.h
├── tests/
│   ├── run.sh       ← 테스트 작성/실행을 위한 스크립트
│   └── data/
│       └── sample.txt ← 예시 입력 파일

```

### 4 사용법

```

make          # 계측 없이 일반 빌드 → build/grep
make cov      # gcov 커버리지 계측 빌드 → build/grep-cov
make reset    # 누적된 커버리지 데이터 삭제
make check    # 현재까지 누적된 커버리지 확인
make report   # 누적된 커버리지 데이터를 지우고, tests/run.sh를 실행한 후 커버리지 출력
make clean    # 빌드 산출물 제거

```

### 5 제출물

‘학번/’ 폴더를 만들어 다음과 같은 구조를 만든 후 .zip 파일로 압축하여 Cyber Campus에 제출합니다.

```

2699999/
├── run.sh          (필수)
├── data/           (입력 파일이 있는 경우)
└── ...

```

#### 5.1 run.sh

다음과 같은 구조를 갖는 run.sh를 작성합니다.

```

#!/bin/bash

# grep 경로는 반드시 $MYGREP으로 대체해서 작성
# 예: $MYGREP -rn "XYZ"

```

```
cd "$(dirname "$0")"

# 1
$MYGREP "hello" data/sample.txt

# 2
$MYGREP -i -n "HELLO" data/sample.txt

# 3
echo "pipe me" | $MYGREP "pipe"
```

### 5.1.1 반드시 지켜야 하는 규칙

- grep 실행 파일 경로는 항상 \$MYGREP으로 씁니다. 채점 시 계측된 바이너리 경로를 이 변수에 넣어 실행하게 됩니다. 절대 경로나 ./grep으로 쓰면 채점되지 않습니다.
- \$MYGREP 호출은 최대 30회. 31번째부터는 무시됩니다.
- 한 줄에 한 호출을 원칙으로 합니다. 파이프()로 입력을 넣는 경우는 한 줄로 봅니다.
- 첫 줄 다음에 cd "\$(dirname "\$0")"를 넣어, 어느 디렉터리에서 실행해도 상대 경로가 맞도록 합니다.

## 5.2 입력 파일

- 검색 대상 텍스트 파일, -f에 넘길 패턴 파일 등은 제출 폴더 안에 넣고 상대 경로로 참조합니다.
- 시스템 파일(/etc/passwd, /usr/share/dict/words 등)에 의존하면 안 됩니다. 채점 환경에 없을 수 있습니다.
- 입력 파일 전체 합계 10MB 이하.

## 6 금지 사항

다음은 테스트 설계 능력이 아니라 채점 시스템을 우회하는 방법 및 채점을 어렵게 만드는 방법이므로 금지합니다. 위반 시 해당 실행은 무효 처리되고, 정도가 심하면 기본 점수에서 감점합니다.

- for, while 등으로 \$MYGREP 실행 횟수를 늘리는 것
- \$MYGREP을 다른 변수에 재할당해 우회하는 것
- 절대 경로 사용, 제출 폴더 밖의 파일 참조
- 네트워크 접근, 파일 생성·삭제 등 부수 효과를 일으키는 명령
- 실행 중 사용자 입력을 기다리는 구성 (파일 인자 없이 stdin을 비워두면 멈춥니다. 이 경우 echo ... | 또는 < 파일로 반드시 입력을 공급할 것)

### 6.1 실행 제한

- 실행 1회당 10초, 전체 스크립트 5분. 초과하면 해당 실행은 중단되며, 해당 실행의 커버리지는 인정되지 않습니다.
- run.sh의 종료 코드는 채점에 영향을 주지 않습니다. grep이 0이 아닌 다른 종료 코드를 반환해도 괜찮습니다. 오히려 그런 입력이 필요할 수도 있습니다.

## 7 커버리지 확인 방법

제출 전에 본인 점수의 근거가 되는 수치를 직접 확인할 수 있습니다.

`make report`

출력 예시:

```
-----
라인 커버리지 : 27.91% of 3271
분기 커버리지 : 13.99% of 3416   <- 채점 지표
-----
```

미커버 코드는 `grep.c.gcov` 에서 '#####' 를 찾아보세요.

채점에 쓰는 지표는 두 번째 줄, 분기 커버리지입니다. 첫 줄의 라인 커버리지를 참고하지 않습니다.

## 8 채점 기준

기본점수 50 : 상대점수 50

- 실행에 아무런 문제가 없다면 기본점수 50%를 부여합니다.
- 기타 문제(스크립트 실행 불가, \$MYGREP 호출 30회 초과, 제출 폴더 구조 위반 등)가 있으면 기본점수 50%에서 감점합니다.
- 최대 30개의 입력 실행이 달성한 분기 커버리지의 순위에 따라 상대점수 50%를 부여합니다.
- 과제를 제출하지 않으면 F이고, 빈 과제를 제출하면 0점입니다.

## 9 정직성

- 과제는 반드시 본인이 직접 설계하고 작성해야 합니다. 다른 사람의 과제를 그대로 제출하거나, 다른 사람의 과제를 참고하여 작성한 경우, 심각한 부정행위로 간주합니다.
- 부정행위가 적발되면 해당 과제는 0점 처리되며, 학칙에 따라 추가적인 조치가 있을 수 있습니다.
- 채점 환경을 조작하려는 시도가 확인되면 0점 처리합니다.